DAY 04 Loops

Phase 2: Beginner

■ Duration	■ Session	■ Phase	■ Level
2 Hours	Day 04 of 30	Beginner	Foundational

What You'll Teach Today

Concept	Description	Time
`for` loop	Iterate over sequences, ranges, and collections	25 min
`while` loop	Repeat while a condition is True	20 min
`range()`	Generate numeric sequences with start/stop/step	15 min
`break` / `continue` / `pass`	Control loop execution flow	15 min
`else` on loops	Else clause that fires when loop isn't broken	10 min
Nested loops	Loops inside loops — patterns and pitfalls	15 min
Wrap-up & Q&A	Review, exercises, next day preview	20 min

SECTION 1 — SESSION OVERVIEW

Session Goal

By the end of this session students will be able to write **for** and **while** loops confidently, use **range()** to generate sequences, apply **break/continue/pass** to control flow, and reason about when each loop type is appropriate.

Learning Outcomes

- Write `for` loops over lists, strings, dictionaries, and ranges
- Write `while` loops with clear termination conditions
- Generate sequences using `range(start, stop, step)`
- Use `break` to exit a loop early and `continue` to skip iterations
- Explain what `pass` does and when to use it
- Recognise and write the `else` clause on a loop
- Construct nested loops and reason about their complexity

Minute-by-Minute Timeline

Time	Activity	Mode
0:00 – 0:05	Welcome & recap of Day 3 control flow	Discussion
0:05 – 0:30	`for` loop — syntax, sequences, `range()`	Live Coding
0:30 – 0:50	`while` loop — conditions & infinite loops	Live Coding
0:50 – 1:05	`break`, `continue`, `pass`	Live Coding
1:05 – 1:15	`else` on loops	Demonstration
1:15 – 1:30	Nested loops — multiplication table demo	Live Coding
1:30 – 1:40	Student exercises (solo)	Practice
1:40 – 1:50	Walk through solutions together	Review
1:50 – 2:00	Wrap-up, checklist, Day 5 preview	Discussion

■ Instructor Note

Loops are where most beginners first get stuck — infinite loops, off-by-one errors, and confusion between for vs while. Budget extra time for the while section. Have students run a deliberately infinite loop and show them Ctrl+C to break it.

SECTION 2 — PREREQUISITES & PREPARATION

Student Prerequisites

Topic	Covered On	Required?
Variables & data types	Day 2	■ Required
if / elif / else control flow	Day 3	■ Required
Comparison & logical operators	Day 3	■ Required
Python installed + VS Code	Day 1	■ Required

Instructor Pre-Session Checklist

- ■ Open VS Code with a fresh `day04_loops.py` file ready
- ■ Have Python REPL open in terminal — students follow along
- ■ Prepare a short list `fruits = ['apple', 'banana', 'cherry']` to iterate over
- ■ Prepare to demo a deliberate infinite loop (while True:) — know Ctrl+C
- ■ Have the multiplication table nested loop demo scripted
- ■ Test that `range()` examples run on Python 3.12

SECTION 3 — STEP-BY-STEP INSTRUCTION GUIDE

Step 1 — The `for` Loop

What to Say:

"A **for loop** lets you visit every item in a sequence — one at a time — and do something with it. Think of it like: *for each egg in the carton, crack it*. Python makes this incredibly readable."

Live Code — Iterating a list:

```
# ■■ for loop over a list ■■
fruits = ['apple', 'banana', 'cherry']

for fruit in fruits:
    print(fruit)

# Output:
# apple
# banana
# cherry
```

Live Code — `range()` basics:

```
# range(stop)  — 0 up to (but NOT including) stop
for i in range(5):
    print(i)          # prints 0 1 2 3 4

# range(start, stop)
for i in range(2, 8):
    print(i)          # prints 2 3 4 5 6 7

# range(start, stop, step)
for i in range(0, 20, 5):
    print(i)          # prints 0 5 10 15

# Counting down
for i in range(10, 0, -1):
    print(i)          # 10 9 8 ... 1
```

Live Code — Iterating strings and dictionaries:

```
# Strings are iterable!
for char in "Python":
    print(char)       # P y t h o n

# Dictionary — iterates keys by default
scores = {"Alice": 95, "Bob": 87, "Carol": 92}
for name in scores:
    print(f"{name}: {scores[name]}")

# .items() for key-value pairs
for name, score in scores.items():
    print(f"{name} scored {score}")
```

■ Verify Students Can

Write a for loop over a list without looking at notes
Use range(5), range(2, 10), range(0, 100, 10) correctly
Iterate over a string character by character

Common Errors — `for` Loop

Error	Cause	Fix
IndentationError	Loop body not indented	4 spaces inside loop
TypeError: 'int' is not iterable	`for i in 5` — int not iterable	Use `range(5)`
Loop body runs zero times	`range(5, 3)` is empty	Check start < stop
Off-by-one: misses last item	Using `range(len(lst)-1)`	Use `range(len(lst))` or `for x in lst`

Step 2 — The `while` Loop

What to Say:

"A **while loop** keeps running as long as a condition stays True. Use it when you don't know in advance how many times you'll loop — for example, keep asking the user for input until they enter something valid."

```
# ■■ Basic while loop ■■
count = 0
while count < 5:
    print(count)
    count += 1      # IMPORTANT: must change the condition!

# ■■ Simulated user input (no real input() in notebooks) ■■
password = "wrong"    # simulate first attempt
while password != "secret":
    print("Try again")   # in real code, prompt here
    password = "secret"  # simulate correct guess
print("Access granted!")
```

Live Demo — Deliberate Infinite Loop:

```
# ■■ DEMO: infinite loop ■■ (press Ctrl+C to stop!)
# while True:
#     print('looping forever...')

# Always ensure the condition can eventually become False
# OR use break to exit (covered next)
```

■■ Common Pitfall

Infinite loops: the condition never becomes False because the loop body doesn't update the variable being tested. Always ask: 'what inside this loop makes the condition eventually False?'

Error	Cause	Fix
Infinite loop	Condition never becomes False	Update condition variable inside body
Loop never runs	Condition is False before starting	Check initial value
Logic error in condition	`while x > 0` but x starts negative	Print x before loop to debug

Step 3 — `break`, `continue`, and `pass`

What to Say:

"These three keywords give you fine-grained control over what happens inside a loop. **break** exits immediately, **continue** skips to the next iteration, **pass** is a placeholder that does nothing — useful when Python requires a statement but you have nothing to put there yet."

```
# ■■ break: exit the loop immediately ■■
numbers = [3, 7, 12, 4, 9, 21]
for n in numbers:
    if n > 10:
        print(f'Found {n}, stopping!')  # Found 12, stopping!
        break

# ■■ continue: skip this iteration, go to next ■■
for n in range(10):
    if n % 2 == 0:    # skip even numbers
        continue
    print(n)           # prints 1 3 5 7 9

# ■■ pass: do nothing placeholder ■■
for n in range(5):
    pass               # skeleton — fill in later
```

Step 4 — `else` on Loops

What to Say:

"Python loops have a surprising feature: an **else** clause. It runs when the loop finishes normally — that is, when it was NOT exited by a **break**. This is incredibly useful for search patterns."

```
# ■■ else fires when loop completes without break ■■
primes = [2, 3, 5, 7, 11]
target = 6

for p in primes:
    if p == target:
        print(f"{target} is prime")   # skipped
        break
else:
    print(f"{target} is NOT in primes list")  # this runs!

# Same pattern with while:
x = 10
while x > 0:
    x -= 3
    if x == 1:
        break
else:
    print("Loop finished without break")  # prints if x never == 1
```

Step 5 — Nested Loops

What to Say:

"You can put a loop inside a loop. The inner loop runs to completion for every single iteration of the outer loop. The classic example is a multiplication table. Be careful: nested loops have **O(n²)** complexity — they can be slow for large inputs."

```
# ■■ Multiplication table ■■
for row in range(1, 6):
    for col in range(1, 6):
        print(f'{row * col:4}', end='')  # 4-wide column
    print()  # newline after each row

#    1   2   3   4   5
#    2   4   6   8  10
#    3   6   9  12  15
#    4   8  12  16  20
#    5  10  15  20  25

# ■■ Nested loop with break only exits inner loop ■■
for i in range(3):
    for j in range(3):
        if j == 1:
            break   # only breaks inner loop!
        print(f"i={i}, j={j}")
```

■ Key Insight

`break` inside a nested loop only exits the INNERMOST loop.
To break out of multiple loops, use a flag variable or restructure into a function.

SECTION 4 — SESSION WRAP-UP

Student End-of-Session Checklist

- ■ I can write a `for` loop over a list, string, and range
- ■ I understand what `range(start, stop, step)` produces
- ■ I can write a `while` loop with a proper exit condition
- ■ I know how to use `break` and `continue`
- ■ I understand what `pass` is for
- ■ I can explain the `else` clause on a loop
- ■ I can write a simple nested loop
- ■ I completed the notebook exercises without peeking at solutions

Key Concepts Recap

Keyword	Purpose	Use When
for x in seq	Iterate over known sequence	Number of iterations is known
while cond	Repeat while condition is True	Number of iterations is unknown
range()	Generate integer sequence	Need indexes or numeric loops
break	Exit loop immediately	Found what you needed
continue	Skip to next iteration	Skip invalid/unwanted values
pass	Placeholder — does nothing	Structuring code skeleton
else (loop)	Runs if loop didn't break	Search patterns

Day 5 Preview — Functions & Scope

Tomorrow we learn to package reusable logic into **functions** using `def`. We'll cover parameters, return values, default arguments, `*args`/`**kwargs`, and the LEGB scope rule. Students should think about repetitive code they've written in Days 2–4 that could be wrapped in a function.

Troubleshooting Notes

Problem	Likely Cause	Solution
Infinite loop locks up notebook	`while` condition never False	Kernel → Interrupt; show Ctrl+C
`for i in range` prints nothing	range is empty (start >= stop)	Print range as list to debug
IndentationError in loop body	Mixed tabs & spaces	Set VS Code to Spaces: 4
break doesn't exit outer loop	break only exits innermost	Use flag variable or function
continue skips too much	condition too broad	Print variables to trace logic

SECTION 5 — QUICK REFERENCE CARD

Loop Syntax Cheat Sheet

```
# ■■ for loop ■■■
```

■ Done When...

Students can write both loop types from scratch without reference
Students correctly predict output for break/continue examples
Notebook exercises are complete and runnable